Iterators

→ for efficient looping
→ better memory management.

```
>>> my_list = [1,2,3,4,5]
    iterator = iter(my_list)
>>> type(iterator)
```

⇒ <class 'list_iterator>

To access the elements,

```
>>> next(iterator)
```

→ returns the next element in the iterator.

⇒ run again, → returns 2nd element.

→ and so on.

At the last element, it will throw an error.

so,

```
>>> try:
        print(next(iterator))
    except Exception as e:
        print('out of elements')
```

⇒ Out of elements

Same thing can be done with strings.

lazy loading will always need ~~not~~ conversion

Generators → simple way to create iterators.

→ uses yield

→ use yield to generate a value, on the fly,
→ doesn't store it on memory.

g →>>>

```
def square(n):
    for i in range(n):
        yield *i**2
a = square(3)
```

→ creates a local variable

→ a generator object.

To access the generator value,

- for loop over iterator (a)
- iterate using next(a)

important things

→ between an iteration, generators do not reset
→ don't store past values
→ don't replay

Alternate way to make a generator.

```
a = (n*n for n in range(3))
```

generator object

Now, print(next(a)) ⟹ 0

print(next(a)) ⟹ 1

print(list(a)) ⟹ [4]

∵ only 4 was left, rest were exhausted.

(*) ~~Gen~~ Generators → where?

→ To access large files without storing it on memory.

(*) ?? Best pythonic way to transpose a matrix?

$$a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$\boxed{a' = list(zip(*a))}$$

converts the tuples into desired matrix.

zips the unpacked lists ~~A~~ into tuples

* = unpacking character (as learnt in ~~dy~~ tuples)

(*) Interview gold